

Cross-Review of Subsystem C

Item	Note
API Design	Clean and consistent. All functions validate null pointers first and return documented error codes. The header clearly maps return values (-1, -2, -3 and below) to categories of failure.
sync_init Error Cleanup	Proper cascading cleanup
Turnstile / Writer Starvation	Correctly implemented.
Permission Matrix	Sensibly chosen.
resource_id Parameter Unused	Unused.
Dead Fields: next_ticket, turn_number	Both fields are initialized in sync_init and sync_destroy but never read or written elsewhere.
sync_destroy Partial Failure	If sem_destroy(&state->resource_access) fails, the function returns early and neither the turnstile nor the mutex is destroyed.
MANAGE Action Has No Corresponding Function	ADMIN is granted MANAGE permission, but there is no sync_begin_manage / sync_end_manage function. A caller can check for MANAGE permission but cannot perform any managed operation through the subsystem.

Overall, Subsystem C is well-structured and the core logic is sound. The API is clean, the return code convention is consistent, and the readers-writers implementation correctly handles the critical concern of writer starvation via the turnstile pattern. The main areas for improvement are largely housekeeping: dead fields from an abandoned design, an incomplete error path in sync_destroy, and an unimplemented MANAGE operation that the API surface implies but does not deliver. The most notable gap is the absence of any concurrent testing or demonstration —

since the entire purpose of this subsystem is synchronization, validating it only in a single-threaded context leaves the actual thread-safety of the implementation unverified. Adding even one multi-threaded test scenario would significantly strengthen confidence in the implementation.